Winter 2021

CE 459: VLSI Algorithmics

Lab 1. Compiling, Debugging, and Executing Design Using Algorithmic C Data-types

<u>Disclaimer:</u> This tutorial was based on original ***Mentor Graphics*** tutorials. Slight modifications were required due to divergences in the versions of the tools and deprecated instructions. All credits go to the authors of the original tutorials.

In this tutorial we will inspect the following topics:

- (1) Use g++ that ships in catapult
- (2) Add include paths to the catapult install tree
- (3) Debug typical errors with algorithmic data types
- (4) Understand left shift behavior of AC data-types

Before we begin with the tutorial...

Remember to check Tutorial 0 to assert how to setup ssh to access the Wilkinson server and configure and run Catapult for the first time. The link is here.

Steps:

0. Access the Wilkinson server with your NetID and open a Terminal.

1. Load the cad tools environment by running command **(1)**:

```
>> source ~/envsetup-catapult.sh
```
(1)

2. Navigate to the Labs directory and then to Lab1 using command **(2)**:

```
>> cd ~/Labs/Lab1
```
(2)

Let's also take a look at the contents of the folder before proceeding by running **(3)**, which should display something like shown below:

```
>> ls -lh
total 240K
-rw-r--r-- 1 mbv6231 ug  855 Oct 28  2018 Makefile
-rwxr-xr-x 1 mbv6231 ug 209K Jan 11 12:01 tb
-rw-r--r-- 1 mbv6231 ug 1012 Oct 28  2018 tb.cpp
-rw-r--r-- 1 mbv6231 ug  791 Oct 28  2018 tb_pod_err.cpp
-rw-r--r-- 1 mbv6231 ug  655 Oct 28  2018 test_chan_assert.cpp
-rw-r--r-- 1 mbv6231 ug  686 Oct 28  2018 test.cpp
-rw-r--r-- 1 mbv6231 ug  583 Oct 28  2018 test.h
-rw-r--r-- 1 mbv6231 ug  678 Oct 28  2018 test_shift_loss.cpp
```
(3)

As you can see we have some source C++ files in this folder (*.cpp/*.h) along with a subfolder called tb (which stands for testbench) and a Makefile.

3. Let's take a look at the Makefile. Makefiles are files which contain a set of directives used by a make build automation tool to generate a target/goal. They are widely used in several languages compilation processes as they provide an easy way to automate compilation and generation of source code [1].

Open the makefile on your favorite text editor (vim/gedit/emacs…) or use command **(4)**.

```
>> gedit Makefile &
```
(4)

The content of the Makefile (which should match the content shown on **Figure 1**), basically points to the include directory of the Mentor Graphics installation directory (MGC_HOME variable) and to the GCC directory. Then it defines 5 different make rules (tb0, tb1, tb1_debug, tb2 and tb3). We will use all of these later on.

Notice that the task of each one of these rules is basically to invoke gcc compiler with a set of source files. So, for instance, tb0 basically runs gcc on files "*tb_pod_err.cpp*" and "*test_chan_assert.cpp*".

```
#Set environment variable 'MGC_HOME' to point to your Catpult install path
#setenv MGC_HOME /tools/catapult/Catapult_Synthesis_10.3-793810/Mgc_home
#or export MGC_HOME=/tools/catapult/Catapult_Synthesis_10.3-793810/Mgc_home

IDIR =$(MGC_HOME)/shared/include

CC=$(MGC_HOME)/bin/g++
CFLAGS=-g

tb0: tb_pod_err.cpp test_chan_assert.cpp
        $(CC) -o tb tb_pod_err.cpp test.cpp -I$(IDIR) $(CFLAGS)
tb1: tb.cpp test_chan_assert.cpp
        $(CC) -o tb tb.cpp test_chan_assert.cpp -I$(IDIR) $(CFLAGS)
        ./tb
tb1_debug: tb.cpp test_chan_assert.cpp
        $(CC) -o tb tb.cpp test_chan_assert.cpp -I$(IDIR) $(CFLAGS)
        $(MGC_HOME)"/bin/gdb" tb
tb2: tb.cpp test_shift_loss.cpp
        $(CC) -o tb tb.cpp test_shift_loss.cpp -I$(IDIR) $(CFLAGS)
        ./tb
tb3: tb.cpp test.cpp
        $(CC) -o tb tb.cpp test.cpp -I$(IDIR) $(CFLAGS)
        ./tb
.PHONY: clean

clean:
        rm -f *.o tb
```

Figure 1. Content of Makefile

4. Let's make the first target. Go to the terminal again (feel free to close the Makefile) and run command **(5)**, which should raise the error shown below.

```
>> make tb0
```

(5)

```
/vol/mentor/Catapult_Synthesis_10.4a/Mgc_home/bin/g++ -o tb
tb_pod_err.cpp test.cpp
-I/vol/mentor/Catapult_Synthesis_10.4a/Mgc_home/shared/include -g
tb_pod_err.cpp: In function ‘int main()’:
tb_pod_err.cpp:27:89: error: cannot pass objects of
non-trivially-copyable type ‘class ac_fixed<3, 3, false,
(ac_q_mode)1u, (ac_o_mode)1u>’ through ‘...’

data0.to_uint(),wrap_behavior.to_uint(),sat_behavior,shift_behavi
or.to_uint());

^
make: *** [Makefile:11: tb0] Error 1
```

Let’s stop a second to take a look at the error. The error is referencing class *“ac_fixed<3, 3, false, (ac_q_mode)qu, (ac_o_mode)1u>”* at line 27 of file “*tb_pod_err.cpp”*.

5. Let’s open the file that raised the error: “*tb_pod_err.cpp*”. The content of this file is shown in **Figure 2**. As you can see, in line 27 we are calling **printf** function. Remember that the compiler was mentioning in the error a variable of type *“ac_fixed<3, 3, false, (ac_q_mode)qu, (ac_o_mode)1u>”* ? Well, if we look at the initialization of variables in the top of the file we see that the only variable of this class is *sat_behavior*.

   We are trying to print this variable in **line 27**, but there is no direct/implicit conversion from ac_fixed to integer (which is the format we are telling the printf function that variable *sat_behavior* should be printed with “*%3d*”). Note that all the other ac data-type variables are using the “*.to_int()*” helper function, but it is missing on the “*sat_behavior*” variable. Thus, this line will always raise an error.

mbv6231@batgirl:Lab1 ⌥⌘1

```
1 #include "test.h"
2
3 #include <stdio.h>
4 #include <fstream>
5 using namespace std;
6
7 int main()
8 {
9   ac_int<4,false> data0; // 4 bit unsigned, 0 to 15
10   ac_fixed<5,5,false> data1;
11   ac_channel<ac_int<4,false> > chan_in;
12   ac_int<3,false> wrap_behavior;
13   ac_fixed<3,3,false,AC_RND,AC_SAT> sat_behavior;
14   ac_int<5,false> shift_behavior;
15   ac_channel<ac_int<4,false> > chan_out;
16
17   for (int i=0; i<16; i++) {
18     if (i>0) {
19       chan_in.write(i);
20     }
21     data0 = i;
22     data1 = i;
23
24     test(data0,data1,chan_in,wrap_behavior,sat_behavior,shift_behavior,chan_out);
25
26     printf("Input = %3d wrap_behavior = %3d, sat_behavior = %3d, shift_behavior = %3d\n",
27            data0.to_uint(),wrap_behavior.to_uint(),sat_behavior,shift_behavior.to_uint());
28   }
29   return 0;
30 }
31
                                                                    1,1          All
```

Figure 2. Content of file *"tb_pod_err.cpp"*

6. This error has been solved in "*tb.cpp*" file. Take a look at the file and see how line 27 has changed.

7. Let's make the second target. Go to the terminal again and run command **(6)**, which should raise another, different error, as shown below.

```
>> make tb1
/vol/mentor/Catapult_Synthesis_10.4a/Mgc_home/bin/g++ -o tb
tb.cpp test_chan_assert.cpp
-I/vol/mentor/Catapult_Synthesis_10.4a/Mgc_home/shared/include -g
./tb
Assert in file
/vol/mentor/Catapult_Synthesis_10.4a/Mgc_home/shared/include/ac_c
hannel.h:248 Read from empty channel
```

(6)

```
tb:
/vol/mentor/Catapult_Synthesis_10.4a/Mgc_home/shared/include/ac_c
hannel.h:177: static void ac_channel<T>::ac_assert(bool, const
char*, int, const ac_channel_exception::code&) [with T =
ac_int<4, false>]: Assertion `0' failed.
make: *** [Makefile:14: tb1] Aborted (core dumped)
```

8. As you can see, as this is a RUNTIME ERROR, we do not know exactly where this error was raised, so we will need to debug it in order to be able to find the problem. Any type of advanced IDE debugger tool could be used here, but let's use gdb here, as it ships with Catapult. Go to the terminal and run command **(7)**, which will make target tb1 in debug mode.

```
>> make tb1-debug
(gdb)
```

(7)

9. Now let's run the code, which should stop once the error is raised. Type command **(8)** in the gdb terminal.

```
(gdb) run
```

(8)

You will see that this command returns the same error mentioned before, referring to "*Reading from empty channel*". Now we can go ahead and start debugging.

10. In this step we will debug our code by using the command "*up*". This command allows us to go back (up) in the code from the point where the assertion (error) failed. We will go up until we find the first call of the function that will give us a clue of what the source of the error might be. Run command up 7 times, as shown in command **(9)** in the gdb terminal.

```
(gdb) up
#1  0x00007ffff7103c35 in abort () from /lib64/libc.so.6

(gdb) up
#2  0x00007ffff7103b09 in __assert_fail_base.cold.0 () from
/lib64/libc.so.6

(gdb) up
#3  0x00007ffff7111de6 in __assert_fail () from /lib64/libc.so.6

(gdb) up
```

(9)

```
#4  0x0000000000401c30 in ac_channel<ac_int<4, false>
>::ac_assert (condition=false,
    file=0x403208
"/vol/mentor/Catapult_Synthesis_10.4a/Mgc_home/shared/include/ac_
channel.h",
    line=248, code=@0x7fffffffdc4c:
ac_channel_exception::read_from_empty_channel)
    at
/vol/mentor/Catapult_Synthesis_10.4a/Mgc_home/shared/include/ac_c
hannel.h:177
177             assert(0);

(gdb) up
#5  0x000000000040182c in ac_channel<ac_int<4, false>
>::fifo::read (this=0x7fffffffdd20)
    at
/vol/mentor/Catapult_Synthesis_10.4a/Mgc_home/shared/include/ac_c
hannel.h:248
248             AC_CHANNEL_ASSERT(size(),
ac_channel_exception::read_from_empty_channel);

(gdb) up
#6  0x0000000000401484 in ac_channel<ac_int<4, false> >::read
(this=0x7fffffffdd20)
    at
/vol/mentor/Catapult_Synthesis_10.4a/Mgc_home/shared/include/ac_c
hannel.h:114
114     T read() { return chan.read(); }

(gdb) up
#7  0x0000000000402ca0 in test (data0=..., data1=...,
chan_in=..., wrap_behavior=...,
    sat_behavior=..., shift_behavior=..., chan_out=...) at
test_chan_assert.cpp:15
15      chan_out.write(chan_in.read());      // will assert if
read of empty channel is attempted
```

Take a look at the message that appeared in the last call of "*up*". Do you see anything that suggests what the error might be? According to this line (line 15 in the *"test_chan_assert.cpp"* file), we should assert if the channel *"chan_in"* is empty before trying to read from it, but we are not doing so. If you open *"test_chan_assert.cpp"* file you will see that this assertion never occurs.

11. Quit dgb by running **(10)**.

```
(gdb) quit
```

(10)

12. The previous error has been fixed in file "*test_shift_loss.cpp*". Let's take a look at it. Opening using the text editor. Its content should be as shown in **Figure 3**.

Take a look at line 15. Do you see how here we are using the "*.available(1)*" method? This returns true only if the channel is available for read. This fixes the previous error.

```
mbv6231@batgirl:Lab1                                                    ⌥⌘1
 1 #include "test.h"
 2
 3 void test(ac_int<4,false> data0,
 4           ac_fixed<5,5,false> data1,
 5           ac_channel<ac_int<4,false> > &chan_in,
 6           ac_int<3,false> &wrap_behavior,
 7           ac_fixed<3,3,false,AC_RND,AC_SAT> &sat_behavior,
 8           ac_int<5,false> &shift_behavior,
 9           ac_channel<ac_int<4,false> > &chan_out)
10 {
11   wrap_behavior = data0;                // will wrap for values > 7
12   sat_behavior = data1;                 // will clamp to 7 for values > 7
13   shift_behavior = data0 << 1;          // will lose MSB after shift
14
15   if (chan_in.available(1)) {           // Only read if channel not empty
16     chan_out.write(chan_in.read());
17   }
18 }
19
~
~
~
                                                                  1,1          All
```

Figure 3. Content of file *"test_shift_loss.cpp"*

13. Let's make the fourth target. Go to the terminal again and run command **(11)**. This type we should not see any errors. Instead, we should see the actual behavior of the source code.

```
>> make tb2

/vol/mentor/Catapult_Synthesis_10.4a/Mgc_home/bin/g++ -o tb
tb.cpp test_shift_loss.cpp
-I/vol/mentor/Catapult_Synthesis_10.4a/Mgc_home/shared/include
-g
./tb
Input =  0 wrap_behavior =  0, sat_behavior =  0,
shift_behavior =  0
Input =  1 wrap_behavior =  1, sat_behavior =  1,
shift_behavior =  2
Input =  2 wrap_behavior =  2, sat_behavior =  2,
```

(11)

```
shift_behavior =  4
Input =  3 wrap_behavior =  3, sat_behavior =  3,
shift_behavior =  6
Input =  4 wrap_behavior =  4, sat_behavior =  4,
shift_behavior =  8
Input =  5 wrap_behavior =  5, sat_behavior =  5,
shift_behavior = 10
Input =  6 wrap_behavior =  6, sat_behavior =  6,
shift_behavior = 12
Input =  7 wrap_behavior =  7, sat_behavior =  7,
shift_behavior = 14
Input =  8 wrap_behavior =  0, sat_behavior =  7,
shift_behavior =  0
Input =  9 wrap_behavior =  1, sat_behavior =  7,
shift_behavior =  2
Input = 10 wrap_behavior =  2, sat_behavior =  7,
shift_behavior =  4
Input = 11 wrap_behavior =  3, sat_behavior =  7,
shift_behavior =  6
Input = 12 wrap_behavior =  4, sat_behavior =  7,
shift_behavior =  8
Input = 13 wrap_behavior =  5, sat_behavior =  7,
shift_behavior = 10
Input = 14 wrap_behavior =  6, sat_behavior =  7,
shift_behavior = 12
Input = 15 wrap_behavior =  7, sat_behavior =  7,
shift_behavior = 14

Channel data =  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15
```

Look at the testbench output

- The input is an ac_int<4,false> 4 bits unsigned and counts from 0 to 15
- Wrap_behavior is and ac_int<3,false> 3 bits unsigned and copies the input. You can see how it “wraps” at 7 back to 0. Just like a wire or register in Verilog
- Sat_behavior is ac_fixed<3,3,false,AC_RND,AC_SAT> 3 bits unsigned with saturation enabled. Note how it clamps the data at 7.
- Shift_behavior gets the input left shifted by 1, which is the same as multiplying by 2. However, note that it only goes to 14 and then back to 0. This is because data0 is an ac_int<4,false> 4 bit unsigned integer. Left shifting will cause the MSB to be lost.

14. Let's open the file *"test.cpp"*. Look at line 13. See how we are now casting "*data0*" variable to 5 bits to avoid the problem we were having with the left shifting before?

```
mbv6231@batgirl:Lab1                                                    ⌥⌘1
 1 #include "test.h"
 2
 3 void test(ac_int<4,false> data0,
 4           ac_fixed<5,5,false> data1,
 5           ac_channel<ac_int<4,false> > &chan_in,
 6           ac_int<3,false> &wrap_behavior,
 7           ac_fixed<3,3,false,AC_RND,AC_SAT> &sat_behavior,
 8           ac_int<5,false> &shift_behavior,
 9           ac_channel<ac_int<4,false> > &chan_out)
10 {
11   wrap_behavior = data0;                // will wrap for values > 7
12   sat_behavior = data1;                 // will clamp to 7 for values > 7
13   shift_behavior = (ac_int<5,false>)data0 << 1; // cast to 5 bits to keep MSB
14
15   if (chan_in.available(1)) {           // Only read if channel not empty
16     chan_out.write(chan_in.read());
17   }
18 }
~
~
~
~
"test.cpp" [dos] 18L, 686C                                          1,1          All
```

Figure 4. Content of file *"test.cpp"*

15. Finally, let's make the fifth and last target. Go to the terminal again and run command **(12)**.

```
>> make tb3

/vol/mentor/Catapult_Synthesis_10.4a/Mgc_home/bin/g++ -o tb
tb.cpp test.cpp
-I/vol/mentor/Catapult_Synthesis_10.4a/Mgc_home/shared/include
-g
./tb
Input =  0 wrap_behavior =  0, sat_behavior =  0,
shift_behavior =  0
Input =  1 wrap_behavior =  1, sat_behavior =  1,
shift_behavior =  2
Input =  2 wrap_behavior =  2, sat_behavior =  2,
shift_behavior =  4
Input =  3 wrap_behavior =  3, sat_behavior =  3,
shift_behavior =  6
Input =  4 wrap_behavior =  4, sat_behavior =  4,
shift_behavior =  8
```

(11)

```
Input =  5 wrap_behavior =  5, sat_behavior =  5,
shift_behavior = 10
Input =  6 wrap_behavior =  6, sat_behavior =  6,
shift_behavior = 12
Input =  7 wrap_behavior =  7, sat_behavior =  7,
shift_behavior = 14
Input =  8 wrap_behavior =  0, sat_behavior =  7,
shift_behavior = 16
Input =  9 wrap_behavior =  1, sat_behavior =  7,
shift_behavior = 18
Input = 10 wrap_behavior =  2, sat_behavior =  7,
shift_behavior = 20
Input = 11 wrap_behavior =  3, sat_behavior =  7,
shift_behavior = 22
Input = 12 wrap_behavior =  4, sat_behavior =  7,
shift_behavior = 24
Input = 13 wrap_behavior =  5, sat_behavior =  7,
shift_behavior = 26
Input = 14 wrap_behavior =  6, sat_behavior =  7,
shift_behavior = 28
Input = 15 wrap_behavior =  7, sat_behavior =  7,
shift_behavior = 30

Channel data =  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15
```

As you can see, the problem has been solved and the code behaves now as expected.

References

[1] Learn makefiles online content. https://makefiletutorial.com/